

AlphaLab

The Master Design, Section by Section

A plain-language companion — read it next to the real document

This guide walks through the Master Design Document in the same order it is written — section 0 to section 22, plus the big decision log — and restates each part in ordinary language. The aim is simple: after reading a section here, you should be able to explain the matching section of the real document to someone else, in your own words, without the jargon. Nothing here replaces the Master Design; it is a translator that sits beside it.

How to use it: keep the Master Design open. When a section number here matches a section number there, you are reading the same idea twice — once in engineering language, once in plain English.

Research / paper-trading only. Not investment advice.

What's inside

The one-page mental model	the two jobs the whole system does
§0–1 Purpose & the honesty philosophy	why 'falsify, don't confirm' drives everything
§1.1 The power reality	why a winner takes years — and that's fine
§1.2 What 'success' means	graded on trustworthiness, not profit
§2 The decision log (D1–D71)	how to read the 71 numbered choices
§3 The two-layer model	inside a strategy vs. between strategies
§4–5 Technology & architecture	the moving parts and how they connect
§6 The daily funnel	how a strategy picks stocks each day
§7 Local math vs. Claude	what's free arithmetic, what uses the AI
§8 Live vs. Candidate	the self-improvement loop
§9 Strategies	the roster and what research says
§10–11 Metrics & the Overfitting Monitor	how skill is measured and self-deception caught
§12 Sizing & safety	position sizes and the guardrails
§13–14 Data & the database	where the numbers come from and live
§15 The screens	what you actually look at
§16 The golden rules	the 31 non-negotiables
§17 What 'improvement' can mean	and the vertical-slice build order
§19 The math, in plain language	the eight statistical ideas
§20 The gap-closure specs	the hardest-to-explain pieces
§21–22 The UI boundary	why any front end is swappable
D71 The arena model	one universe per league

START HERE

The one-page mental model

Before any section makes sense, hold these two facts in your head. The entire document is built on them.

Fact one — the system does two separate jobs. Job A is judging a single strategy: does this one rule have real skill? Job B is choosing between strategies: given several rules, which one do I trust with the (fake) money, and how do I spread money across them? Almost every confusing passage clears up the moment you ask 'is this about judging one strategy, or choosing among several?'

Fact two — the enemy is self-deception, not the market. It is easy to build something that looks brilliant on old data and fails on new. So the whole design is a set of traps for your own optimism: realistic trading costs always charged, a crowd of mindless random strategies as the yardstick, and a rule that no result counts until the data is large enough to support it.

IN PLAIN TERMS

The lab is a referee, not a fortune-teller. It is designed to tell you the truth about your ideas — usually the unwelcome truth that an idea can't beat luck — rather than to make a number go up.

SECTIONS 0–1

Purpose & the honesty philosophy

These opening sections state what the lab is for and the seven principles it never breaks.

The lab exists to answer one question rigorously: does a strategy have a real edge, and if so, when and why? Every principle that follows is a way of not fooling yourself. The headline one is **build to falsify, not to confirm**: success isn't 'it showed a profit,' it's 'I can trust what the lab tells me' — including the very likely verdict that most ideas don't beat simply buying and holding after costs.

The other principles are all guardrails: forward paper results are the only judge (never a backward-looking backtest); judge by expectancy and risk-adjusted return, never win rate; comparisons must be fair before they can be honest; always show what the data can and cannot say; every piece of data has a named source, a validation check, and a fallback; and the AI only reads text — all the number-crunching is plain, free, deterministic code.

IN PLAIN TERMS

A tool that can only confirm your ideas is a mirror. A tool that can prove them wrong is a laboratory. AlphaLab is deliberately built to be the second kind.

SECTION 1.1

The power reality — why a winner takes years

This is the most important reality check in the whole document, and the one most worth being able to explain.

The realistic prize from a sensible large-company strategy is small — roughly 1 to 3 percent a year of genuine skill above the market. The uncomfortable arithmetic is that proving an effect that small, to a scientific standard, needs a very large amount of data — on the order of a decade or two of live results. Even after a full year, the smallest effect the data can honestly confirm is several times bigger than the prize itself.

So the everyday verdict will be **'too early to judge,'** and that is the system working, not failing. But there is an important asymmetry the document is careful about: the lab can be *fast* at two things even though confirmation is slow. It can quickly flag a strategy that is actively *worse* than random (anti-predictive), and it can quickly say a strategy is *indistinguishable from random* — a real, honest, fast answer even though it isn't a 'winner.'

IN PLAIN TERMS

Confirming a real edge is a decades-long job, so 'too early to tell' is the normal, honest answer for years. What the lab *can* do quickly is spot the genuine losers and say plainly when an idea is no better than a coin flip.

SECTION 1.2

What 'success' actually means

If the lab isn't graded on making money, what is it graded on?

It is graded on properties of the *laboratory itself*: does every comparison honestly show its margin of error? Does it catch bad strategies fast? Does it say 'indistinguishable from random' out loud instead of hiding behind 'too early' forever? Are the mindless random strategies promoted no more often than pure chance would predict? Is there no data leakage? Does every year-old strategy get a 'what is this, really?' breakdown?

A rising equity curve appears nowhere on that scorecard. If the lab produces trustworthy 'no edge,' 'too early,' and 'this is just repackaged momentum' verdicts, it is succeeding — even if it never crowns a winner.

IN PLAIN TERMS

Think of grading a referee, not a player. A good referee is judged on making correct, fair calls — not on the final score. The lab is judged the same way.

SECTION 2

The decision log (D1 through D71)

This is the longest part of the document — a numbered list of every design choice ever made. Here's how to read it without drowning.

Each entry has a number (D-something), a one-line decision, and a reason. The numbers are permanent labels: the rest of the document — and the whole build — says things like 'per D36' instead of re-explaining. You don't need to memorize them; you need to know they exist so that when a section cites 'D43,' you know where to look it up.

A few carry most of the weight and are worth knowing by name: **D36** (the random 'control populations' — the yardstick), **D37** (Arena Replay — the practice mode that tests the machinery), **D57/D58** (the swappable-UI boundary), **D63** ('indistinguishable from random' as a real verdict), and **D71** (multiple arenas). The numbers climb over time because each review pass appended new ones rather than rewriting history — so the *why* behind every choice is never lost.

IN PLAIN TERMS

The decision log is the project's memory. Every rule elsewhere points back to a D-number here, like footnotes pointing to their source. You look them up; you don't recite them.

SECTION 3

The two-layer mental model

This section draws the single most useful picture in the document.

The outer layer — between strategies. Several candidate strategies each run on their own fake-money account. A 'promotion gate' periodically asks whether any candidate clearly beats the current trusted ('Live') one. Usually the answer is 'too early.' Separately, an 'allocator' spreads money across them in small, careful tilts.

The inner layer — inside one strategy. How a single rule turns market data into a decision: it scores stocks, holds them for a declared period, and exits by a declared policy. If a strategy blends several signals, the combiner is a 'meta-classifier' — but that only shows up much later in the build.

IN PLAIN TERMS

Zoom out: which strategy do I trust, and how do I split money? Zoom in: how does one strategy actually decide? Keep those two zoom levels separate and the rest of the document stops being confusing.

Technology & architecture

The parts list, and a map of how they connect.

The build is C#.NET with a single SQLite database file, a web-based screen layer, and Claude used only as a text-reader. The clever structural choice is splitting the system into separate programs that each do one job. The **Worker** is the only program allowed to write to the database — it runs each evening, catches up on any days it missed, and quits. The **Api** is a thin doorway every screen talks through. The **Web** app is just one interchangeable front end behind that doorway.

Why bother? Because it means your computer doesn't need to be always on (the Worker just catches up when you next open it), and the screens can be swapped for a phone app later without touching the delicate math underneath.

IN PLAIN TERMS

One program does the thinking and writing (evenings only), one is a service window, and the screens are wallpaper you can replace. Your laptop can be asleep half the week and the lab still stays correct.

SECTION 6

The daily decision funnel

How every strategy picks its stocks each day — the same six steps for all of them.

First, **eligibility**: the same shared pool of tradeable stocks for everyone, so any difference in results is really about the strategy. Then **scoring**, which is where strategies differ — momentum and mean-reversion hand back almost opposite lists. Then **selection** (only positive-scoring names — never pad the list to look busy), **portfolio** decisions (open/add/trim/exit by the strategy's own exit rules), **sizing and safety**, and finally **orders**.

One quietly important rule: decide at today's close, but fill at tomorrow's open. A strategy never gets to trade on a price it couldn't actually have acted on — a common way backtests cheat without meaning to.

IN PLAIN TERMS

Every strategy runs the same assembly line; they only differ at the 'which stocks look good' step. And nothing trades on a price it couldn't have really caught — no peeking at today's close to buy at today's close.

SECTION 7

Local math vs. Claude — the division of labor

What is free arithmetic, and what actually uses the AI (and costs money).

Almost everything — scoring, sizing, costs, profit-and-loss, the statistics, the verdicts — is plain local code. It costs nothing and gives the same answer every time. Claude's job is narrow: read the day's news into a tidy summary, and act as an on-demand research assistant (write a bull/bear brief, play devil's advocate about a suspicious result). The daily 'sentiment score' from the news is explicitly treated as experimental — the design fully expects it might turn out to be worth roughly nothing, and has an honest A/B test to find out.

Cost is controlled several ways: the news that enters the prompt is capped and de-duplicated, the routine reads run at half price in batches, and there's a hard daily budget with graceful degradation if it's hit.

IN PLAIN TERMS

The number-crunching is free, exact code. The AI is a hired reader and sparring partner on the side — useful, strictly budgeted, and never trusted with the actual arithmetic.

SECTION 8

Live vs. Candidate — the self-improvement loop

How the lab is supposed to get better over time.

One strategy is 'Live' (the trusted one). Others are 'Candidates' on probation, trading in parallel with their own fake money. On a set schedule — not daily, to avoid cherry-picking a lucky day — the gate checks whether a candidate genuinely beats Live by more than the data's margin of error, with the overfitting monitor's blessing. If so, it gets promoted; if the promotion later regresses, it's reverted.

But binary promotion is the rare, ceremonial event. The *main* improvement mechanism is the allocator making small, continuous, logged tilts in how money is spread — because gentle tilts are the honest thing to do when the evidence is weak, which it usually is.

IN PLAIN TERMS

Rather than dramatic 'you're hired / you're fired' moments, the lab mostly improves by nudging money gradually toward whatever is looking better — small, reversible, well-documented steps.

SECTION 9

The strategies, and what research says

The starting roster and the honest expectations attached to it.

The baselines come first: buy-and-hold (two versions) and the random control crowds. Then price-only strategies — momentum (buy recent winners), mean-reversion (buy recent overreactions), low-volatility (prefer calmer stocks). Fundamental strategies (value, quality) are deferred because trustworthy point-in-time fundamental data is genuinely hard to get and is gated behind a validation step.

Four research truths shape all of it: edges **decay** as they get crowded; **diversifying across different edges** is the real advantage; **robustness beats optimization** (a knife-edge 'perfect' setting is usually just noise); and a cautious, long-only, large-company version of a published strategy keeps perhaps a third to a half of the textbook premium.

IN PLAIN TERMS

Start with the boring benchmarks and the coin-flippers, then add a few well-understood price strategies. Expect modest, fading edges — and treat any strategy that looks too perfect as a warning sign, not a triumph.

SECTIONS 10–11

Metrics & the Overfitting Monitor

How skill is measured, and how the lab polices itself against self-deception.

The headline metric is **beta-adjusted alpha** — return above what the strategy's market exposure alone would explain. That matters because a cautious low-volatility strategy would lose every raw-return comparison in a bull market while working exactly as designed; adjusting for market exposure is what makes the comparison fair. Every comparison is shown next to its margin of error, and the gate never acts inside that margin.

The **Overfitting Monitor** is eight independent alarms — things like 'did it degrade from backtest to live?', 'is it separated from the random crowd?', 'is its edge decaying?', 'do its different metrics disagree with each other?' A 'Suspect' rating vetoes any promotion regardless of profit, and sustained decay auto-retires a strategy. The thresholds aren't guessed — they're calibrated in practice mode first.

IN PLAIN TERMS

Skill is measured as return beyond what market exposure alone buys — always shown with its error bars. And a bank of eight lie-detector tests runs constantly; if one trips, a strategy can't be promoted no matter how good its profit looks.

SECTION 12

Sizing, safety & portfolio construction

How big each position is, and the safety rails.

The default sizing gives calmer stocks more weight and jumpier stocks less, then scales the whole book to a target risk level — using a proper estimate of how the stocks move *together* (fifteen correlated mega-caps are not fifteen independent bets). A more aggressive sizing method (Kelly) is deliberately held back until much later, because its inputs can't be estimated reliably early on.

The guardrails all 'fail closed': a minimum score, a maximum position size, a cap on total predicted risk, a limit on how many positions at once, a cooldown after exits, regime-based halts, and a cap on how much of a stock's daily volume you can pretend to trade. If a required input is missing, the order is rejected — never guessed.

IN PLAIN TERMS

Give steadier stocks bigger slices, risky ones smaller, and never let the whole plate get too heavy. Every safety rule errs toward doing nothing rather than doing something on incomplete information.

SECTIONS 13–14

Data sourcing & the database

Where the numbers come from, and why one simple database file is enough.

Every data feed is named, validated against a second source, and given a fallback — prices, dividends, which stocks are in the index (and were, historically), sector labels, news, and academic factor data. Two ideas prevent whole classes of silent bugs: a **permanent internal ID** for each company (so a ticker change like FB→META doesn't look like one company vanishing and another appearing), and **versioned, never-deleted prices** (corrections arrive as new versions, so any past day can be reproduced exactly as it was known then).

The database is a single SQLite file, and the document explains at length why that's plenty: there's exactly one writer, the heavy statistics happen in memory rather than in the database, and the largest table is comfortably under a couple of gigabytes for years. No fancier database would add anything this design uses.

IN PLAIN TERMS

Trust no single data source blindly — cross-check everything, and never overwrite history. Give each company a permanent internal ID so name changes don't confuse the books. And one humble file is genuinely enough here.

SECTION 15

The screens

What you actually look at — and the honesty rules baked into every one.

There's a screen for the Live strategy, one ranking all strategies, a 'why this trade' explainer, an overfitting-health view, allocation, trades, risk, regimes, data-health, a research journal, and more. The guiding principle is 'make the honest reading the easy reading': verdicts outrank raw returns, every comparison carries its margin of error, every strategy shows where it ranks against the random crowd, and practice-mode results are visually quarantined so they can never masquerade as real ones.

Crucially, none of the screens compute any of this. The honesty — what to dim, what chip to show, what to quarantine — is worked out once in the code underneath and handed to the screen as ready-made data. That's what lets any front end be swapped in without risking a screen that quietly shows a number the system meant to hide.

IN PLAIN TERMS

The screens are designed so the truthful reading is also the most eye-catching one. And they're 'dumb' on purpose — all the honesty is decided underneath, so no screen can accidentally flatter a bad strategy.

SECTION 16

The golden rules

Thirty-one one-line non-negotiables — the behavioral contract for the whole build.

These are the rules that, if broken, are bugs rather than matters of taste: forward results are the only judge; costs are always on; alpha always means the market-adjusted kind; never delete or overwrite a price; fail closed on missing inputs; every strategy is ranked against its matched random crowd; secrets never enter the code repository; and so on.

You don't need them memorized. You need to know they're the checklist the builder (and Claude Code) is held to, and that each one traces back to a hard-won lesson in the decision log.

IN PLAIN TERMS

A short list of 'never do this' commandments, each earned from a specific way past systems have fooled their builders. They're the fence around the cliff edge.

SECTION 17

What 'improvement' can mean — and the build order

An honest section about what can and can't get better, plus how to actually build it.

What can improve: your *selection* (converging on robust, diversified strategies), the *allocation* across them, and — the real product — *your own understanding*, including trustworthy negative results. What can't reliably improve: the raw returns of a fixed strategy (edges decay). A system whose performance climbs forever is a red flag for a leak, not a triumph.

The build order is a deliberate 'thin vertical slice': aim straight for the practice-mode sign-off (where the machinery is proven) using a smaller stock universe and operating through the plain service interface, treating the pretty screens as a parallel task due later. The biggest real risk isn't the statistics — it's abandoning a months-long solo build, so the phase gates and the honest progress log are the actual defense.

IN PLAIN TERMS

The lab's returns won't magically climb forever — what grows is your understanding and your judgment. And the smart way to build it is straight down the middle to the proof point, leaving the decoration for later.

SECTION 19

The mathematics, in plain language

The document's own math primer — eight ideas, each explained from intuition. Here they are in one breath each.

Paired testing: compare two strategies on the *same days* so the market's shared mood cancels out, leaving only their disagreement — far easier to judge. **Margin of error (MDE):** the smallest effect the data can honestly detect yet; anything smaller is 'unjudgeable,' not 'absent.' **Newey–West:** a correction for the fact that consecutive days aren't independent evidence, so the margin of error isn't overstated.

Beta and alpha: separate 'just the market, scaled' from genuine skill. **Deflated Sharpe:** if you try enough strategies, the luckiest looks great by chance — so discount for how many you tried. **Population percentile:** where a strategy ranks among hundreds of matched random ones — a model-free 'could dumb luck have done this?' **Shrinkage / Ledoit–Wolf:** an honest compromise between trusting noisy data and a simple assumption. **Square-root cost impact:** big orders move the price against you, roughly with the square root of their size.

IN PLAIN TERMS

Every statistical trick in the lab has one job: stop a lucky result from looking like a skilled one. This section explains each one with a plain-English intuition before any formula — it's the friendliest part of the document.

SECTION 20

The gap-closure specs (the hard-to-explain pieces)

This section holds the trickiest, most technical specifications. Two are worth understanding because they're genuinely subtle.

Why 'indistinguishable from random' had to become a real verdict (20.8): because the random crowd pays the same trading costs as your strategy, a merely mediocre strategy sits right in the *middle* of the crowd forever — the crowd can never prove it's bad, only that it's ordinary. So the lab says exactly that, out loud, with a day count ('no separation from 200 random strategies after 417 days'), instead of hiding behind 'too early' indefinitely.

Why the 'planted' test strategies are specified so carefully (20.9): to calibrate its alarms, the lab plants fake strategies with a known edge and sees how long they take to detect. But a fake edge that trickles in evenly is unrealistically easy to spot; a real edge arrives in lumpy streaks and takes far longer. If they'd used the easy fake, every honest real strategy would look 'suspect' for years. So the plants are deliberately realistic and lumpy, and there's a mandatory check comparing the two.

IN PLAIN TERMS

These are the fiddly bits. The key insight: a cost-paying strategy that's merely mediocre can never be *proven* bad — it just blends into the random crowd — so the lab learned to say 'this is indistinguishable from luck' as a first-class, honest answer.

SECTIONS 21–22

The UI boundary — why any front end is swappable

The last two sections explain the doorway between the screens and the system, and why it matters.

Every screen talks to the system through one service (the Api) using plain web requests. Behind it, all the 'honesty' — which numbers to dim, which verdict to show, what to quarantine — is computed once and shipped to the screen as finished data ('read-models'). The screen's only job is to display what it's handed, faithfully.

The payoff: you can replace the web screens with a phone app or a different framework and it's physically impossible for the new front end to show a number the system meant to hide — because the front end never does that reasoning in the first place. The honesty lives in one tested place, shared by every possible screen.

IN PLAIN TERMS

Think of the screens as plugged into a single, honest wall socket. Whatever you plug in gets the same carefully-checked electricity — no screen can rewire the truth, because the truth is decided before it reaches the plug.

The arena model — one universe per league

The most recent big idea, and the one you're actively building toward.

An **arena** is one complete, self-contained practice competition built around one pool of stocks — the S&P; 500 today, perhaps the Russell 2000 (smaller companies) later. Each arena is fully separate: its own database file, its own random crowd, its own cost model, its own record book. They share the software, never the results.

Why the hard separation? Because ranking a large-company strategy against a small-company one compares numbers produced under different cost models and different random crowds — the same meaningless mismatch as declaring a featherweight boxer 'better' than a heavyweight. So the screens can show two arenas side by side, but never merge them into one ranking. And the plan is deliberate: build and prove the S&P; 500 arena first, then add a second only once the machine is proven — adding one is copying a working setup and changing a couple of settings, not a rewrite.

IN PLAIN TERMS

Each stock universe gets its own separate league, with its own scoreboard. You never rank a big-company strategy against a small-company one — different leagues. Prove the first league works, then clone the machine for the next.

The one sentence to remember. AlphaLab is a private, honest referee for stock-picking ideas: it runs each idea with realistic costs against a crowd of mindless random ones, tells you the truth about whether there's real skill (usually: 'not yet' or 'no better than luck'), and is built so that no screen, no shortcut, and no wishful thinking can make a bad idea look good.

Companion to MASTER_DESIGN_v1.9. Read the two together — this guide for the intuition, the Master Design for the exact specification. Research / paper-trading only. Not investment advice.